Planificación e implementación de bases de datos relacionales en PostgreSQL

Breve descripción:

Este componente formativo proporciona los fundamentos teóricos, arquitectónicos y prácticos necesarios para concebir, planificar y desplegar infraestructuras de almacenamiento de datos robustas. El aprendiz asimilará desde la abstracción de reglas de negocio en modelos conceptuales hasta la instalación física y estructuración técnica de objetos relacionales utilizando el motor de código abierto PostgreSQL.

Septiembre 2026

Tabla de contenido

Introducción 4

1. Arquitectura y fundamentos de los sistemas de gestión de bases de datos 5

1.1. Introducción y conceptos de sistemas manejadores de bases de datos (DBMS) 5

1.2. Drivers y storage engines 10

1.3. Herramientas gráficas de gestión y administración 15

2. Planificación estratégica, licenciamiento y diseño conceptual 19

2.1. Planificación: el diseño conceptual de una base de datos 19

3. Entornos de despliegue, infraestructura y configuración inicial 23

3.1. Ciclo de instalación del servidor e instancias locales 23

3.2. Despliegue en la nube y proveedores de instancias PostgreSQL 29

4. Tipos de datos y modelado estructural físico (DDL) 33

4.1. Tipos de datos y estructuración física de objetos 33

4.2. Construcción de la base de datos mediante Lenguaje de Definición de Datos (DDL) 36

4.3. Restricciones de integridad y relaciones físicas 39

Síntesis 42

Glosario 43

Referencias bibliográficas 46

Créditos 47

Introducción

En la actualidad, los datos constituyen el activo estratégico más valioso de las organizaciones. Sin embargo, su aprovechamiento exige sistemas capaces de garantizar la seguridad, la integridad y la disponibilidad de la información. En este contexto, este componente formativo introduce al estudio de los Sistemas Manejadores de Bases de Datos (DBMS, por sus siglas en inglés, Database Management Systems) mediante PostgreSQL, un motor de bases de datos de código abierto, orientado a objetos y ampliamente adoptado en la industria por su estabilidad, potencia y alta escalabilidad.

El propósito de este componente consiste en desarrollar las competencias necesarias para transformar las necesidades del sector productivo en estructuras digitales eficientes. Para ello, se aborda la ingeniería de datos desde la etapa de planificación, iniciando con la comprensión de la arquitectura interna de PostgreSQL y sus esquemas de licenciamiento. Posteriormente, el proceso avanza hacia el diseño conceptual de la información y la estructuración técnica de repositorios relacionales orientados a entornos corporativos.

Con el fin de alcanzar este propósito, el proceso de formación adopta un enfoque eminentemente práctico que permite desarrollar planes de trabajo organizados y alineados con escenarios reales. Mediante el uso de interfaces gráficas como pgAdmin y entornos de ejecución locales o en la nube, se elaborarán scripts utilizando el Lenguaje de Definición de Datos (DDL, por sus siglas en inglés, Data Definition Language). Este proceso permitirá construir tablas, definir relaciones e implementar restricciones de integridad que garanticen la consistencia, confiabilidad y solidez de las soluciones informáticas desarrolladas.

1. Arquitectura y fundamentos de los sistemas de gestión de bases de datos

La comprensión de los fundamentos estructurales de un motor de bases de datos constituye el primer paso para garantizar el éxito y la viabilidad de cualquier solución informática en el sector productivo. Esta unidad temática permite comprender los conceptos arquitectónicos esenciales, los niveles de abstracción de la información y el funcionamiento interno de PostgreSQL.

Asimismo, proporciona las bases para entender cómo se gestionan las conexiones concurrentes, cómo opera el almacenamiento físico de los datos y de qué manera las herramientas gráficas optimizan las tareas de administración bajo un enfoque técnico, seguro y orientado a la alta disponibilidad.

1.1. Introducción y conceptos de sistemas manejadores de bases de datos (DBMS)

Los sistemas de información contemporáneos demandan mecanismos que superen la simple persistencia de archivos dispersos en un disco duro. En el entorno corporativo actual, la integridad de los procesos operativos depende de la capacidad para centralizar, estandarizar y proteger el acceso a los activos lógicos de la organización. Para responder a este desafío, surgen arquitecturas de software especializadas que abstraen la complejidad del almacenamiento físico y proporcionan interfaces de programación seguras y predecibles.

Un Sistema Manejador de Bases de Datos (DBMS, por sus siglas en inglés, Database Management System) representa una infraestructura crítica que unifica las operaciones de creación, consulta, actualización y mantenimiento de la información. La implementación de esta tecnología permite mitigar los riesgos asociados con los

accesos simultáneos y la corrupción de archivos, además de fortalecer la administración centralizada de los datos.

Definición, evolución e importancia de los DBMS en el sector productivo

A continuación, se presenta un pódcast sobre el Sistema Manejador de Bases de Datos (DBMS), sus principales funciones y su importancia en la gestión eficiente, segura y confiable de la información dentro de las organizaciones modernas:

Transcripción pódcast. Sistema Manejador de Bases de Datos (DBMS)
Mujer: Hace algún tiempo muchas aplicaciones manejaban la información directamente en archivos planos. **Hombre:** Y cuando varias personas necesitaban trabajar sobre esa información, aparecían los problemas: datos duplicados, información que no coincidía y poca capacidad para controlar las modificaciones. **Mujer:** Para resolver ese caos se diseñó el Sistema Manejador de Bases de Datos o DBMS: una capa especializada de software que se ubica entre la aplicación y el almacenamiento para organizar la información. **Hombre:** A partir de ese punto, el sistema controla quién puede acceder a los datos, registra las operaciones y ayuda a mantener la información íntegra y disponible. **Mujer:** Imaginen a contabilidad cerrando mes, logística despachando pedidos y talento humano actualizando procesos, todos consultando la misma base de datos.

Hombre: El reto está en que todos puedan trabajar al mismo tiempo sin afectar la información. El DBMS permite que múltiples usuarios trabajen de forma simultánea, aplicando reglas automáticas para que nadie sobrescriba ni altere los registros de los demás.

Mujer: Conforme la plataforma crece, esa capacidad sostiene la operación. El sistema procesa transacciones en tiempo real y genera los reportes que necesitan las distintas áreas.

Hombre: Los datos son uno de los activos estratégicos más valiosos de una organización. Por eso perder o alterar información crítica puede afectar directamente la operación.

Mujer: Un buen desarrollo se demuestra en la arquitectura de datos que sostiene la operación por dentro.

Hombre: El diseño del proyecto debe arrancar garantizando cómo se van a organizar y proteger los datos desde el primer día.

Mujer: Las aplicaciones cambian, crecen o se reemplazan, pero la información que sostiene a la empresa necesita mantenerse segura, disponible y bajo control.

El modelo relacional: tablas, tuplas, atributos y la abstracción de datos

El modelo relacional, propuesto por el científico informático Edgar F. Codd, se fundamenta en la teoría matemática de conjuntos y la lógica de predicados. Este paradigma organiza la información en estructuras bidimensionales denominadas tablas o relaciones. Cada tabla está conformada por un conjunto de tuplas (filas o registros),

que representan instancias individuales de una entidad, y por atributos (columnas o campos), que describen las propiedades o características de cada una de ellas.

Uno de los principios fundamentales del modelo relacional es la abstracción de datos, la cual permite separar la representación física de la información de su estructura lógica y de la forma en que los usuarios interactúan con ella. Gracias a este principio, el sistema divide sus responsabilidades en diferentes niveles, facilitando el mantenimiento, la seguridad y la independencia entre los componentes de la base de datos. La abstracción de datos comprende tres niveles claramente diferenciados, cuyas características se describen a continuación:

- **Nivel físico.** Describe cómo se almacenan los datos en los discos, los bloques de memoria y las estructuras de archivos de bajo nivel. Corresponde a la organización física de la información.
- **Nivel lógico o conceptual.** Define qué datos se almacenan en la base de datos y cuáles son las relaciones existentes entre ellos. En este nivel se establecen las tablas, las restricciones de integridad y la estructura lógica del sistema.
- **Nivel de visión o externo.** Representa las diferentes formas en que los usuarios o las aplicaciones acceden a la información. Permite abstraer parte de la base de datos con fines de simplificación o seguridad, por ejemplo, mediante el uso de vistas.

La separación entre estos niveles constituye uno de los principios fundamentales de los sistemas manejadores de bases de datos modernos, ya que proporciona independencia entre el almacenamiento físico, la organización lógica de la información y las necesidades de acceso de los usuarios y las aplicaciones.

Arquitectura Cliente-Servidor nativa y el ciclo de vida del proceso central postgres

PostgreSQL se define como un Sistema de Gestión de Bases de Datos Relacionales Orientado a Objetos (ORDBMS, por sus siglas en inglés, Object-Relational Database Management System) que implementa, de forma nativa, una arquitectura Cliente-Servidor. En este modelo, el sistema se divide en dos componentes que interactúan mediante un protocolo de comunicación de red: el cliente, encargado de la interfaz de usuario y del envío de solicitudes, y el servidor, responsable del procesamiento, almacenamiento y validación de las instrucciones SQL.

El núcleo operativo del servidor corresponde a un proceso maestro del sistema operativo denominado postgres. Este proceso administra la inicialización del servidor, coordina las conexiones de los clientes y supervisa la creación de procesos independientes para atender cada sesión. Su funcionamiento sigue un ciclo de vida claramente definido:

- **Inicialización y escucha.** Al iniciar el servicio, el proceso maestro postgres se carga en memoria, asigna los segmentos de memoria compartida y entra en estado de escucha sobre un puerto de red específico, que de forma predeterminada corresponde al 5432.
- **Creación de procesos (forking).** Cuando una aplicación cliente, como un sistema de facturación o una interfaz gráfica, solicita una conexión, el proceso maestro valida las credenciales de acceso. Si la autenticación es correcta, ejecuta una operación de fork, mediante la cual crea un proceso hijo independiente (backend process) dedicado exclusivamente a atender esa conexión.

- **Aislamiento operativo.** La existencia de procesos independientes garantiza que, si uno de ellos presenta una falla ocasionada por una consulta incorrecta o un error de memoria, los demás procesos y el proceso maestro continúen funcionando sin interrupciones. Esta arquitectura constituye una de las principales razones de la estabilidad y confiabilidad de PostgreSQL en entornos empresariales.
- **Finalización de la conexión.** Cuando el cliente finaliza la sesión o la conexión expira por inactividad, el proceso hijo asociado termina su ejecución de manera segura y libera los recursos de memoria utilizados durante la sesión.

Esta arquitectura basada en procesos independientes permite administrar múltiples conexiones concurrentes de forma segura, optimizar el uso de los recursos del sistema y fortalecer la disponibilidad del servicio, características que han consolidado a PostgreSQL como uno de los motores de bases de datos más utilizados en aplicaciones corporativas.

1.2. Drivers y storage engines

La eficiencia de una solución tecnológica depende de la solidez de los mecanismos de comunicación entre las aplicaciones y la base de datos, así como de la capacidad del motor para gestionar el almacenamiento de la información. Un sistema difícilmente puede considerarse de alto rendimiento si sus capas de comunicación generan latencia o si los mecanismos de persistencia afectan la disponibilidad durante periodos de alta demanda.

Controladores de conectividad estandarizada corporativa: JDBC, ODBC y alternativas modernas

Para que una aplicación desarrollada en cualquier lenguaje de programación pueda conectarse a un servidor de bases de datos, ejecutar instrucciones del Lenguaje de Definición de Datos (DDL, por sus siglas en inglés, Data Definition Language) o del Lenguaje de Manipulación de Datos (DML, por sus siglas en inglés, Data Manipulation Language), y procesar los resultados obtenidos, requiere un componente de software denominado controlador o driver.

Los controladores proporcionan Interfaces de Programación de Aplicaciones (API, por sus siglas en inglés, Application Programming Interface), las cuales estandarizan la comunicación entre las aplicaciones y el motor de base de datos, independientemente de la tecnología utilizada para desarrollar el sistema. Entre los controladores más utilizados en entornos corporativos se encuentran los siguientes:

- **JDBC (Java Database Connectivity).** Es el controlador estándar del ecosistema Java. Permite establecer conexiones con PostgreSQL, ejecutar consultas, utilizar sentencias preparadas (Prepared Statements) para reducir el riesgo de ataques por inyección SQL y administrar de forma eficiente grupos de conexiones (connection pools) en servidores de aplicaciones.
- **ODBC (Open Database Connectivity).** Constituye un estándar ampliamente utilizado para facilitar la interoperabilidad entre aplicaciones y motores de bases de datos. Es frecuente en entornos Windows y en aplicaciones desarrolladas en C++ y C#, además de herramientas de análisis de datos y suites ofimáticas.

Controladores nativos modernos. Diversos lenguajes incorporan bibliotecas optimizadas que se comunican directamente con el protocolo nativo de PostgreSQL (Frontend/Backend Protocol v3.0). Entre ellas se encuentran PDO_PGSQL para PHP, pg para Node.js (JavaScript y TypeScript) y bibliotecas como psycopg2 o asyncpg para Python, ampliamente utilizadas en desarrollo web, ciencia de datos y aplicaciones de alto rendimiento.

La disponibilidad de múltiples controladores permite integrar PostgreSQL con diferentes plataformas tecnológicas sin modificar el funcionamiento interno del motor de bases de datos, favoreciendo la interoperabilidad, la portabilidad y el desarrollo de aplicaciones empresariales escalables.

Arquitectura de almacenamiento unificada e impacto del mecanismo MVCC

A diferencia de otros sistemas gestores que delegan el almacenamiento físico en motores intercambiables con comportamientos diferentes, PostgreSQL implementa una arquitectura de almacenamiento unificada y altamente integrada. Uno de los pilares que garantiza la consistencia de los datos y el rendimiento de las transacciones es el Control de Concurrencia Multi-Versión (MVCC, por sus siglas en inglés, Multi-Version Concurrency Control).

En los sistemas tradicionales, cuando una transacción ejecuta una operación de actualización o eliminación sobre un registro, el motor bloquea físicamente la fila o incluso la tabla para impedir que otros usuarios accedan a datos inconsistentes mientras finaliza la operación. Este enfoque reduce el rendimiento en entornos con alta concurrencia debido a la espera generada entre las transacciones.

PostgreSQL resuelve esta limitación mediante el mecanismo MVCC, basado en un principio fundamental: las operaciones de lectura no bloquean las operaciones de escritura y las operaciones de escritura no bloquean las operaciones de lectura. Gracias a este enfoque, múltiples usuarios pueden consultar y modificar la información de manera simultánea sin afectar significativamente el desempeño del sistema. El funcionamiento de MVCC se fundamenta en varios mecanismos que trabajan de forma coordinada para garantizar la consistencia y el acceso concurrente a los datos:

- **Aislamiento mediante instantáneas (snapshots).** Cuando una transacción inicia una consulta, PostgreSQL genera una instantánea lógica del estado de la base de datos en ese momento. De esta forma, la consulta trabaja sobre una versión consistente de la información, aunque otras transacciones continúen realizando modificaciones.
- **Creación de versiones de tuplas.** Cuando una instrucción UPDATE modifica una fila, PostgreSQL no sobrescribe el registro original. En su lugar, marca la tupla existente como obsoleta y crea una nueva versión con los datos actualizados, registrando la información necesaria para controlar su visibilidad.
- **Visibilidad condicional.** Mientras la transacción permanece activa, los demás usuarios continúan accediendo a la versión anterior de la fila. La nueva versión solo estará disponible cuando la transacción confirme los cambios mediante la instrucción COMMIT.
- **Proceso autovacuum.** Las versiones antiguas permanecen temporalmente almacenadas en disco. El proceso automático autovacuum identifica estos registros obsoletos, recupera el espacio que ocupan para reutilizarlo en

futuras operaciones y actualiza las estadísticas utilizadas por el optimizador de consultas.

La secuencia de funcionamiento del mecanismo MVCC puede resumirse en cuatro etapas consecutivas:

- **Paso 1. Generación de la instantánea (`snapshot`).** La transacción inicia y obtiene una instantánea consistente de los datos existentes en ese momento.
- **Paso 2. Creación de nuevas versiones.** Cuando se modifica una fila, la versión anterior se marca como obsoleta y se genera una nueva versión con la información actualizada.
- **Paso 3. Control de visibilidad.** Las demás transacciones continúan accediendo a la versión anterior hasta que la transacción que realizó la modificación ejecute COMMIT.
- **Paso 4. Ejecución de `autovacuum`.** El proceso automático elimina las versiones obsoletas desde el punto de vista lógico, recupera espacio de almacenamiento y mantiene actualizadas las estadísticas del sistema.

La arquitectura basada en MVCC constituye uno de los principales factores que diferencian a PostgreSQL de otros sistemas gestores de bases de datos, ya que permite mantener altos niveles de concurrencia, reducir los bloqueos entre transacciones y optimizar el rendimiento en aplicaciones empresariales con múltiples usuarios accediendo simultáneamente a la información.

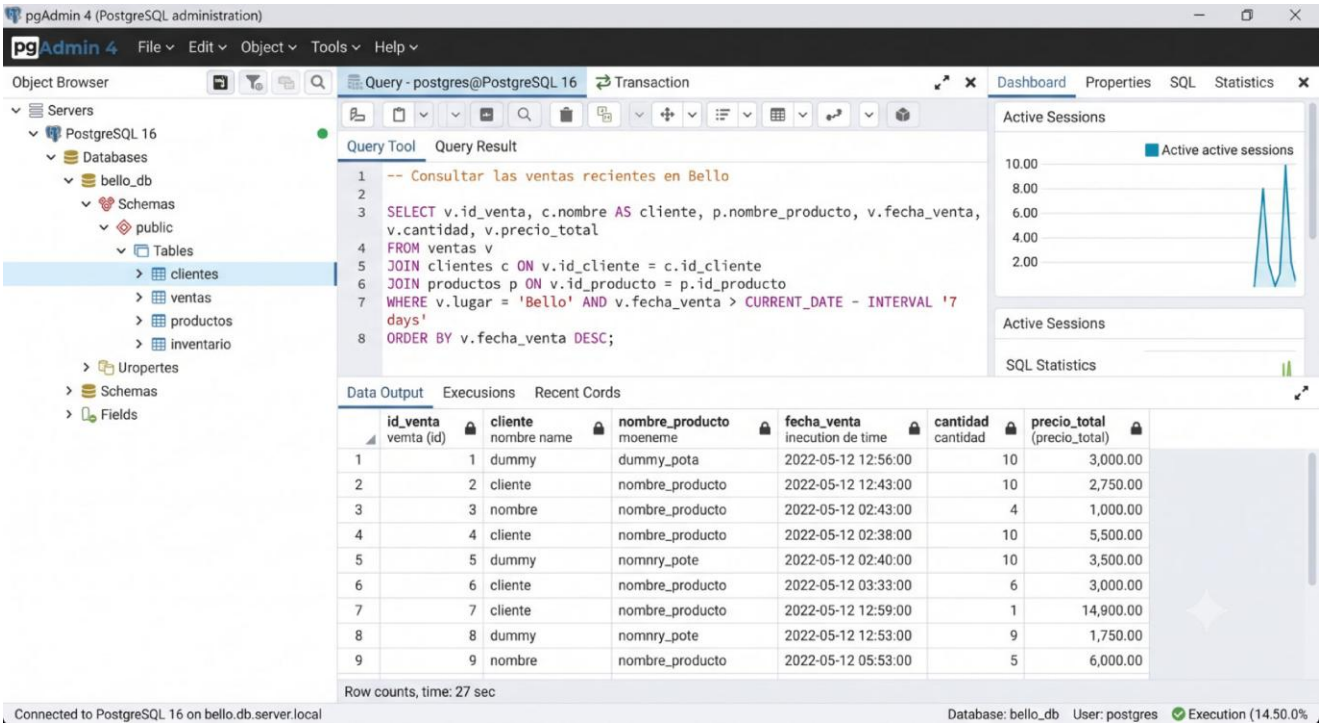
1.3. Herramientas gráficas de gestión y administración

La administración eficiente de un servidor corporativo requiere un equilibrio entre el uso de la línea de comandos y las herramientas de interfaz gráfica. Mientras la consola proporciona un control detallado y facilita la automatización de tareas administrativas, las aplicaciones gráficas agilizan el diseño de estructuras, el diagnóstico del sistema y el análisis del rendimiento mediante entornos interactivos.

Administración visual avanzada con el entorno web centralizado pgAdmin

El dominio de la interfaz de línea de comandos (psql) constituye una competencia fundamental para la administración de PostgreSQL debido a su rapidez y capacidad de automatización. No obstante, las herramientas con Interfaz Gráfica de Usuario (GUI, por sus siglas en inglés, Graphical User Interface) incrementan la productividad durante las etapas de diseño, depuración, monitoreo y administración de la base de datos. La herramienta oficial desarrollada para PostgreSQL es pgAdmin, un entorno de administración que integra funciones de desarrollo, monitoreo y gestión mediante una interfaz gráfica intuitiva.

Figura 1. Entorno gráfico de pgAdmin



pgAdmin está desarrollado sobre una arquitectura web moderna, por lo que puede ejecutarse como aplicación de escritorio o implementarse de manera centralizada en un servidor para permitir el acceso controlado de múltiples administradores y desarrolladores a través de un navegador web. Entre las funcionalidades más relevantes de esta herramienta se encuentran las siguientes:

- **Herramienta de consultas (Query Tool).** Proporciona un editor de código SQL con autocompletado, resaltado de sintaxis y opciones de formateo automático para facilitar la escritura y ejecución de consultas.
- **Análisis del plan de ejecución.** Permite interpretar la estrategia utilizada por el optimizador de PostgreSQL para ejecutar una consulta, identificando operaciones como escaneos secuenciales, uso de índices y diferentes tipos de uniones, lo que facilita la optimización del rendimiento.

- **Panel de rendimiento (Dashboard).** Presenta indicadores en tiempo real sobre sesiones activas, transacciones por segundo, utilización de memoria caché, acceso a disco y estado de los bloqueos existentes en el servidor.
- **Asistentes para la administración de objetos.** Facilitan la creación y administración de usuarios, permisos, restricciones, esquemas y copias de seguridad mediante asistentes gráficos que reducen la necesidad de escribir instrucciones SQL complejas.

Gracias a estas funcionalidades, pgAdmin constituye una herramienta ampliamente utilizada para administrar PostgreSQL tanto en entornos de desarrollo como en ambientes de producción.

Utilidades de escritorio y herramientas alternativas ligeras para entornos de desarrollo

El ecosistema de PostgreSQL ha contado históricamente con diversas herramientas de administración. Entre ellas se encuentran pgAccess y Database Master, aplicaciones de escritorio diseñadas para administrar esquemas y manipular datos en equipos con recursos de hardware limitados. Aunque actualmente su uso ha disminuido frente a herramientas más modernas, su estudio permite comprender la evolución de las interfaces de administración y la interacción con el catálogo interno del sistema.

En los entornos actuales de desarrollo, los equipos de ingeniería de software suelen complementar o sustituir el uso de pgAdmin por herramientas compatibles con múltiples motores de bases de datos. Estas aplicaciones proporcionan una experiencia unificada para administrar diferentes plataformas desde un mismo entorno de trabajo.

Las principales alternativas utilizadas en proyectos de desarrollo se describen a continuación:

- **DBeaver.** Herramienta de código abierto basada en Eclipse que utiliza JDBC (Java Database Connectivity) para conectarse a diferentes motores de bases de datos. Permite administrar instancias locales y remotas mediante una interfaz altamente configurable.
- **DataGrip.** Entorno desarrollado por JetBrains orientado a profesionales de bases de datos. Incorpora autocompletado inteligente, análisis estático de consultas, navegación entre objetos y herramientas avanzadas para el desarrollo y mantenimiento de bases de datos relacionales.

La disponibilidad de diferentes herramientas de administración permite seleccionar la alternativa más adecuada de acuerdo con las características del proyecto, las necesidades del equipo de desarrollo y el entorno tecnológico en el que se implementará la solución.

2. Planificación estratégica, licenciamiento y diseño conceptual

El éxito en la construcción de una solución informática no depende únicamente de la capacidad para desarrollar código, sino también de la rigurosidad con la que se planifica la estructura lógica del sistema y se analiza su viabilidad técnica, legal y económica. Antes de implementar una base de datos en un servidor, es indispensable abstraer con precisión las reglas del negocio y representarlas mediante un modelo independiente de la tecnología.

2.1. Planificación: el diseño conceptual de una base de datos

La fase conceptual constituye el fundamento analítico de todo proyecto de gestión de la información. Modificar la estructura de una base de datos cuando el sistema ya se encuentra en producción representa uno de los reprocesos más complejos y costosos, debido a los riesgos asociados con la migración de datos y la pérdida de consistencia.

Por esta razón, la planificación estratégica requiere el empleo de modelos estandarizados que faciliten la comunicación entre los responsables del negocio y el equipo de desarrollo, permitiendo representar de manera clara los elementos que conformarán la solución tecnológica.

Técnicas de abstracción de reglas de negocio: el Modelo Entidad-Relación (MER)

El diseño conceptual constituye un proceso de traducción mediante el cual la información obtenida de una organización se transforma en un modelo gráfico formal y libre de ambigüedades. El Modelo Entidad-Relación (MER), propuesto por Peter Chen

en 1976, continúa siendo uno de los estándares más utilizados para representar la estructura semántica de los datos (Silberschatz, Korth y Sudarshan, 2002).

Las técnicas de abstracción permiten identificar únicamente aquellos elementos que poseen relevancia para el sistema de información, descartando aspectos del entorno que no aportan valor al modelo conceptual. Este proceso facilita representar de forma precisa la estructura de la organización antes de iniciar el diseño de la base de datos.

La identificación de las reglas de negocio suele apoyarse en un análisis lingüístico de entrevistas, documentos y requerimientos. En términos generales, los sustantivos se convierten en entidades, mientras que los verbos representan las relaciones entre ellas. Una interpretación incorrecta de estas reglas puede afectar la integridad referencial del sistema y generar inconsistencias durante las etapas posteriores del desarrollo.

Determinación de entidades, atributos, claves primarias y especificación de reglas de cardinalidad

El Modelo Entidad-Relación (MER) se compone de varios elementos, cada uno con una función específica dentro de la representación conceptual de los datos:

- **Entidades.** Representan objetos del mundo real con existencia propia dentro de la organización, ya sean tangibles, como un vehículo o un aprendiz, o intangibles, como una matrícula o una venta (Elmasri y Navathe, 2021).
- **Atributos.** Corresponden a las propiedades o características que describen una entidad. Pueden clasificarse como simples, compuestos o multivalorados.

- **Clave primaria (Primary Key - PK).** Es el atributo, o conjunto mínimo de atributos, que identifica de forma única cada registro de una entidad. Debe cumplir dos condiciones fundamentales: no admitir valores nulos (NOT NULL) y garantizar la unicidad (UNIQUE).
- **Reglas de cardinalidad.** Definen la cantidad de entidades que pueden asociarse entre sí dentro de una relación. Se clasifican en relaciones de uno a uno (1:1), uno a muchos (1) y muchos a muchos (N) (Silberschatz, Korth y Sudarshan, 2002). Estas reglas orientan posteriormente la definición de claves foráneas en el modelo relacional.

La correcta identificación de estos componentes permite construir un modelo conceptual consistente y facilita la transición hacia las etapas de diseño lógico y físico de la base de datos. Como aplicación de estos conceptos, se presenta un caso correspondiente a la fase de planificación de un sistema de gestión de recaudo para una empresa de transporte:

- **Abstracción conceptual.** Se analizan las operaciones de la empresa y se identifican dos entidades principales: VEHÍCULO (Placa, Modelo y Capacidad) y CONDUCTOR (Cédula, Nombre y Licencia).
- **Identificación de la clave primaria.** Se establece el atributo Placa como clave primaria de la entidad VEHÍCULO, ya que identifica de manera única cada vehículo de la flota y no admite valores nulos.
- **Definición de las cardinalidades.** El análisis de las reglas de negocio determina que un vehículo puede ser conducido por diferentes conductores y que un conductor puede operar distintos vehículos. Esta situación

corresponde a una relación muchos a muchos (N), por lo que será necesaria una entidad asociativa durante el diseño físico de la base de datos.

- **Validación de la viabilidad.** La organización aprueba la implementación de PostgreSQL al verificar que su modelo de licenciamiento permite desplegar el sistema en los 180 terminales de la flota sin generar costos adicionales por licencias de conexión, favoreciendo la sostenibilidad económica del proyecto.

La planificación conceptual constituye una etapa determinante dentro del ciclo de desarrollo de una base de datos, ya que permite representar correctamente las reglas del negocio, reducir reprocesos durante la implementación y garantizar que el modelo físico responda a las necesidades reales de la organización.

Figura 2. Modelos E-R normalizado



3. Entornos de despliegue, infraestructura y configuración inicial

Una vez consolidados los fundamentos arquitectónicos y el modelo conceptual de los datos, el proceso de formación avanza hacia la implementación de la infraestructura tecnológica. La instalación y configuración adecuada de un servidor de bases de datos constituye una competencia esencial, ya que un entorno incorrectamente parametrizado puede afectar el rendimiento, la disponibilidad y la seguridad de toda la solución informática.

3.1. Ciclo de instalación del servidor e instancias locales

El despliegue local de un motor de bases de datos constituye el punto de partida habitual en los procesos de desarrollo de software. Este entorno proporciona un control completo sobre la infraestructura, facilita las actividades de depuración, pruebas unitarias y simulación de cargas de trabajo, además de eliminar la dependencia de redes externas durante las etapas iniciales del desarrollo.

Sin embargo, una instalación destinada a ambientes corporativos requiere una planificación más rigurosa que una instalación predeterminada. La preparación del sistema operativo, la configuración del almacenamiento y la administración de los servicios representan factores determinantes para garantizar la estabilidad y el desempeño del servidor.

Preparación de entornos operativos, dependencias del sistema y sistemas de archivos optimizados

La instalación de PostgreSQL en un entorno empresarial requiere una evaluación previa del sistema operativo. Entre las plataformas más utilizadas se encuentran

distribuciones Linux como Rocky Linux, Ubuntu Server y Red Hat Enterprise Linux.

Antes de iniciar el proceso de instalación mediante el gestor de paquetes correspondiente, es recomendable preparar el entorno de ejecución. Una de las prácticas más importantes consiste en crear un usuario exclusivo del sistema operativo, habitualmente denominado postgres, evitando que el servicio se ejecute con privilegios del usuario root, ya que esta configuración representa un riesgo para la seguridad del sistema.

La infraestructura de almacenamiento también influye de manera directa en el rendimiento del servidor. La selección del sistema de archivos determina, entre otros aspectos, la velocidad de escritura del mecanismo Write-Ahead Logging (WAL) y la capacidad de recuperación ante fallos inesperados. Las principales recomendaciones para la infraestructura de almacenamiento se resumen a continuación:

- **Sistema operativo.** Utilizar distribuciones Linux orientadas a servidores, como Rocky Linux, Ubuntu Server o Red Hat Enterprise Linux, por su estabilidad y soporte empresarial.
- **Usuario del servicio.** Crear un usuario dedicado denominado postgres para ejecutar el motor de bases de datos sin privilegios de superusuario.
- **Sistema de archivos.** Emplear sistemas de archivos como EXT4 o XFS, configurados para optimizar las operaciones de escritura síncrona.
- **Infraestructura de almacenamiento.** Implementar arreglos RAID 10 combinados con unidades SSD o NVMe cuando se requiera un alto número de operaciones de entrada y salida por segundo (IOPS).

La adecuada preparación del entorno operativo constituye la base para obtener un servidor estable, seguro y con un desempeño acorde con las necesidades de aplicaciones empresariales.

Inicialización del clúster de base de datos (initdb) y gestión del servicio en entornos locales

Una vez instaladas las dependencias necesarias, el siguiente paso consiste en inicializar el clúster de bases de datos mediante la utilidad initdb, incluida de forma nativa en PostgreSQL. Este procedimiento también puede ejecutarse mediante asistentes o scripts incorporados en los instaladores de cada plataforma.

Durante la inicialización se crea la estructura básica del servidor, incluyendo el catálogo del sistema, los directorios internos de almacenamiento como base, pg_wal y pg_tblspc, el usuario administrador inicial (postgres) y la configuración de la codificación de caracteres utilizada por las bases de datos del clúster.

Posteriormente, el control del servicio se delega al administrador de servicios del sistema operativo, como systemd en Linux, desde donde es posible iniciar, detener, reiniciar y verificar el estado del servidor. Los comandos esenciales para administrar el ciclo de vida del servicio se presentan en el siguiente recuadro:

Transcripción código. Comandos esenciales para administrar el ciclo de vida del servicio
Comandos estándar para la administración del servicio PostgreSQL systemctl start postgresql # Inicializa el motor y el proceso maestro de escucha

```
systemctl stop postgresql     # Detiene el servicio de forma segura liberando memoria

systemctl restart postgresql  # Aplica un reinicio completo del clúster de datos

systemctl status postgresql   # Audita el estado actual y los últimos eventos del servicio
```

El dominio de estas utilidades facilita la administración cotidiana del servidor y constituye una competencia fundamental para la operación de entornos de desarrollo y producción.

Instalación en Windows

Además de los entornos Linux, PostgreSQL dispone de instaladores oficiales para el sistema operativo Windows, los cuales automatizan gran parte del proceso de instalación y configuración inicial. A continuación, se presenta un recurso audiovisual que explica paso a paso el procedimiento para instalar PostgreSQL en Windows, realizar la configuración inicial y ejecutar los primeros comandos mediante la consola psql.

Video 1. Instalación y primeros comandos en psql



Enlace de reproducción del video

Síntesis del video. Base de datos con PostgreSQL
El video explica el proceso básico para descargar, instalar, comprobar y comenzar a usar PostgreSQL en Windows. Primero, se indica que PostgreSQL funciona de forma natural en entornos Linux, pero que también puede instalarse fácilmente en Windows. La descarga se realiza desde el sitio oficial de PostgreSQL, aunque el instalador para Windows es proporcionado por EDB (EnterpriseDB). Se selecciona la versión disponible para sistemas de 64 bits y se ejecuta el instalador. Durante la instalación se presentan varios componentes. El más importante es PostgreSQL Server, que corresponde al motor o servidor de la base de datos. También se incluye pgAdmin, una herramienta gráfica que facilita la administración de las

Síntesis del video. Base de datos con PostgreSQL
bases de datos, además de Stack Builder y las herramientas de línea de comandos. Aunque pgAdmin no es indispensable para que PostgreSQL funcione, resulta útil para administrarlo visualmente. El asistente solicita una contraseña para el usuario PostgreSQL, configura por defecto el puerto 5432 y permite mantener la configuración regional del equipo. Una vez finalizada la instalación, se puede verificar su funcionamiento desde los servicios de Windows, comprobando que el servicio correspondiente a PostgreSQL se encuentre en estado de ejecución. Después se introduce el uso de PSQL, la interfaz de línea de comandos de PostgreSQL. Para conectarse localmente se utilizan el servidor localhost, el puerto 5432, la base de datos predeterminada postgres, el usuario postgres y la contraseña establecida durante la instalación. Finalmente, el video muestra algunos comandos básicos. En PSQL, los comandos propios de la herramienta comienzan con backslash (\)**; por ejemplo, \l permite listar las bases de datos disponibles. También se pueden ejecutar instrucciones SQL directamente, como CREATE DATABASE, recordando que las sentencias SQL deben finalizar con **punto y coma (;). Como ejemplo, se crea una nueva base de datos y posteriormente se comprueba su existencia mediante el listado de bases de datos.

Síntesis del video. Base de datos con PostgreSQL
En síntesis, el video sirve como una introducción práctica para instalar PostgreSQL en Windows, reconocer sus principales componentes, comprobar que el servidor esté activo y realizar las primeras operaciones desde PSQL.

Este procedimiento permite disponer de un entorno local completamente funcional para iniciar el desarrollo, la administración y las pruebas de bases de datos utilizando PostgreSQL.

3.2. Despliegue en la nube y proveedores de instancias PostgreSQL

La computación en la nube ha transformado la forma en que las organizaciones implementan y administran su infraestructura tecnológica. Aunque el conocimiento de los entornos locales continúa siendo indispensable, gran parte de las soluciones actuales se desarrollan sobre arquitecturas administradas conocidas como Bases de Datos como Servicio (DBaaS, por sus siglas en inglés, Database as a Service).

Este modelo traslada al proveedor la administración del hardware, las actualizaciones del sistema operativo, los mecanismos de respaldo y otras tareas de infraestructura, permitiendo que los equipos de desarrollo concentren sus esfuerzos en el diseño del modelo de datos, el desarrollo de aplicaciones y la optimización de consultas.

Proveedores en la nube

Diversos proveedores ofrecen planes gratuitos que facilitan el aprendizaje, la experimentación y el desarrollo de prototipos sin necesidad de adquirir infraestructura

propia. Estas plataformas permiten crear instancias de PostgreSQL administradas y acceder a ellas mediante conexiones remotas seguras. Entre las alternativas más utilizadas se encuentran las siguientes:

- **Neon.** Implementa una arquitectura `serverless` para PostgreSQL que separa los recursos de cómputo del almacenamiento. Su plan gratuito permite crear instancias de manera rápida y suspende automáticamente los recursos de procesamiento cuando no existe actividad, optimizando el consumo de infraestructura.
- **Supabase.** Plataforma de desarrollo construida sobre PostgreSQL que, además del motor de base de datos, incorpora servicios de autenticación de usuarios, almacenamiento de archivos y generación automática de API a partir del modelo de datos.
- **Render.** Plataforma en la nube que permite desplegar bases de datos PostgreSQL administradas junto con aplicaciones web. Sus planes gratuitos resultan adecuados para proyectos de formación, pruebas de integración y validación de aplicaciones conectadas a bases de datos relacionales.

La disponibilidad de estos servicios facilita el desarrollo de proyectos sin necesidad de administrar servidores físicos, reduciendo el tiempo requerido para poner en funcionamiento una infraestructura de bases de datos.

Configuración de cadenas de conexión remotas, parámetros de latencia y herramientas de acceso seguro

Para establecer comunicación con una base de datos alojada en la nube, es necesario utilizar una cadena de conexión (`connection string`) o una URL de

conexión. Este parámetro reúne la información necesaria para que el controlador de la aplicación local identifique y establezca comunicación con el servidor remoto.

Un ejemplo de cadena de conexión es el siguiente:

postgresql://usuario:contraseña@host-remoto.cloud.com:5432/nombre_base_datos?sslmode=require

Cada uno de los elementos de la cadena de conexión cumple una función específica:

Tabla 1. Componentes de una cadena de conexión a PostgreSQL

Componente	Descripción
usuario	Nombre de la cuenta con permisos para acceder a la base de datos.
contraseña	Credencial utilizada para autenticar al usuario.
host-remoto.cloud.com	Dirección del servidor donde se encuentra alojada la instancia de PostgreSQL.
5432	Puerto de comunicación utilizado por PostgreSQL de forma predeterminada.
nombre_base_datos	Base de datos a la que se establecerá la conexión.
sslmode=require	Obliga a utilizar una conexión cifrada mediante los protocolos SSL/TLS, garantizando la protección de la información durante su transmisión.

Además de la configuración de la conexión, el despliegue en la nube incorpora aspectos relacionados con el rendimiento de la comunicación. Uno de ellos es la latencia de red, entendida como el tiempo que tarda la información en desplazarse desde el cliente hasta el servidor remoto y regresar con la respuesta correspondiente.

Por razones de seguridad, los proveedores de servicios administrados requieren el uso de conexiones cifradas mediante SSL/TLS, con el fin de proteger la información transmitida a través de Internet. Para administrar estas conexiones es habitual utilizar herramientas como pgAdmin o DBeaver, las cuales permiten registrar las credenciales de acceso, validar la comunicación con el servidor y supervisar el comportamiento de las consultas en entornos remotos.

La comprensión de estos elementos permite establecer conexiones seguras y eficientes con instancias de PostgreSQL alojadas en la nube, favoreciendo el desarrollo de aplicaciones distribuidas y la administración de bases de datos en infraestructuras modernas.

4. Tipos de datos y modelado estructural físico (DDL)

La transición del diseño conceptual hacia la implementación física constituye una de las etapas más importantes en el desarrollo de una base de datos relacional. La eficiencia, el rendimiento y la integridad de una solución informática dependen, en gran medida, de la correcta selección de los tipos de datos y de la definición de restricciones que garanticen la consistencia de la información almacenada.

4.1. Tipos de datos y estructuración física de objetos

La selección del tipo de dato para cada columna constituye una decisión técnica que influye directamente en el consumo de almacenamiento, el rendimiento de las consultas y la integridad de la información. En bases de datos que administran grandes volúmenes de registros, una elección inadecuada incrementa el tamaño de los archivos, reduce la eficiencia de la memoria caché y afecta el tiempo de respuesta de las aplicaciones.

Clasificación de tipos de datos básicos: numéricos, caracteres, temporales y booleanos

PostgreSQL incorpora un amplio conjunto de tipos de datos que permiten almacenar la información de acuerdo con sus características y optimizar el uso de los recursos del sistema. Los principales grupos de tipos de datos se describen a continuación:

Tabla 2. Categorías de tipos de datos en PostgreSQL

Categoría	Tipos de datos	Características y uso
Numéricos	SMALLINT, INTEGER, BIGINT, NUMERIC,	Los tipos enteros almacenan valores sin decimales utilizando 2, 4 u 8 bytes según su capacidad. NUMERIC y DECIMAL son

Categoría	Tipos de datos	Características y uso
	DECIMAL, REAL, DOUBLE PRECISION	apropiados para operaciones financieras que requieren precisión exacta, mientras que REAL y DOUBLE PRECISION se utilizan en cálculos científicos donde se admite representación en coma flotante.
Caracteres	CHAR(n), VARCHAR(n) y TEXT	CHAR(n) almacena cadenas de longitud fija; VARCHAR(n) admite longitud variable con un límite definido, y TEXT permite almacenar cadenas de longitud variable sin límite práctico, siendo apropiado para textos extensos.
Temporales	DATE, TIME, TIMESTAMP y TIMESTAMP WITH TIME ZONE (TIMESTAMPTZ)	Permiten almacenar fechas, horas o ambas simultáneamente. TIMESTAMPTZ resulta especialmente útil en aplicaciones distribuidas, ya que conserva la referencia de la zona horaria y facilita la sincronización entre diferentes ubicaciones geográficas.
Booleanos	BOOLEAN	Almacena valores lógicos mediante los estados TRUE, FALSE y NULL, este último cuando el dato es desconocido o no se ha registrado.

La selección adecuada del tipo de dato contribuye a optimizar el almacenamiento, mejorar el rendimiento de las consultas y garantizar que cada atributo represente correctamente la información del dominio de negocio.

Estructuras de almacenamiento (JSONB y ARRAY) y comportamiento técnico ante valores predeterminados (DEFAULT) o nulos

Además de los tipos de datos tradicionales, PostgreSQL incorpora estructuras especializadas que amplían las posibilidades de modelado dentro de una base de datos relacional. El tipo JSONB permite almacenar documentos en formato JSON utilizando una representación binaria optimizada, lo que facilita la indexación de sus atributos y

mejora el rendimiento de las búsquedas. Esta característica permite combinar información estructurada y semiestructurada dentro de una misma base de datos.

Por su parte, el tipo ARRAY permite almacenar colecciones de valores del mismo tipo en una única columna. Esta funcionalidad resulta útil cuando se requiere representar listas simples sin necesidad de crear tablas adicionales. Durante el diseño físico de las tablas también es necesario definir el comportamiento de las columnas cuando una instrucción de inserción no proporciona un valor o cuando la información es desconocida.

- **JSONB.** Almacena documentos JSON en formato binario optimizado, permitiendo indexar atributos internos y realizar consultas eficientes sobre datos semiestructurados.
- **ARRAY.** Permite almacenar múltiples valores homogéneos dentro de una misma columna, reduciendo la necesidad de crear estructuras adicionales para representar listas simples.
- **DEFAULT.** Asigna automáticamente un valor predeterminado cuando una instrucción INSERT no especifica un dato para una columna. Puede utilizar valores constantes o funciones del sistema, como CURRENT_TIMESTAMP o nextval().
- **NULL.** Representa la ausencia de un valor o la existencia de un dato desconocido. De manera predeterminada, las columnas aceptan valores nulos, salvo que se restrinja este comportamiento mediante la condición NOT NULL.

La correcta combinación de tipos de datos, estructuras de almacenamiento y restricciones permite construir modelos físicos más eficientes, facilita la integridad de la

información y optimiza el comportamiento de la base de datos en entornos de alta demanda.

4.2. Construcción de la base de datos mediante Lenguaje de Definición de Datos (DDL)

Una vez definida la estructura conceptual de la base de datos y seleccionados los tipos de datos apropiados, el siguiente paso consiste en implementar el modelo físico mediante instrucciones SQL. Esta etapa permite materializar el diseño elaborado durante la planificación y crear los objetos que conformarán la base de datos.

El Lenguaje de Definición de Datos (DDL, por sus siglas en inglés, Data Definition Language) corresponde al conjunto de instrucciones del estándar SQL destinadas a crear, modificar y eliminar objetos dentro del catálogo del sistema. A través de estas sentencias es posible construir la estructura lógica del servidor y administrar la evolución del esquema de la base de datos.

Sentencias fundamentales de creación, modificación y eliminación estructural (CREATE, ALTER y DROP)

El ciclo de vida de los objetos que conforman una base de datos se administra mediante un conjunto reducido de instrucciones DDL, cada una con una función específica dentro del proceso de construcción y mantenimiento del esquema. Las principales sentencias se describen a continuación:

- **CREATE.** Crea y registra nuevos objetos dentro del catálogo del sistema, como bases de datos, tablas, vistas, índices o secuencias. Durante la creación de una tabla se definen su nombre, las columnas, los tipos de datos y las restricciones iniciales.

- **ALTER.** Modifica la estructura de un objeto existente sin eliminar la información almacenada. Permite agregar o eliminar columnas, cambiar tipos de datos, renombrar objetos e incorporar nuevas restricciones de integridad conforme evolucionan los requerimientos del sistema.
- **DROP.** Elimina de forma definitiva un objeto del catálogo del sistema, junto con la información asociada cuando corresponde. Su utilización requiere especial precaución, ya que la operación puede resultar irreversible si no existe un mecanismo de respaldo.

El uso adecuado de estas sentencias permite construir, mantener y adaptar la estructura de una base de datos durante todo el ciclo de vida de una aplicación, garantizando que el modelo físico evolucione de acuerdo con las necesidades de la organización.

Transcripción de código. Ejemplo estructurado de sintaxis DDL básica en PostgreSQL

```
CREATE TABLE ejemplo_sena (
  id_registro SERIAL,
  codigo_modulo VARCHAR(15) NOT NULL,
  fecha_creacion TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

Configuración del cotejamiento (Collation) y codificación de caracteres para la internacionalización de datos

Durante la creación de una base de datos mediante la instrucción CREATE DATABASE o al definir columnas de tipo texto, es necesario configurar parámetros que determinan la forma en que PostgreSQL almacena, interpreta y ordena la información. Entre los más importantes se encuentran la codificación de caracteres (character encoding) y el cotejamiento (collation), los cuales garantizan el correcto funcionamiento de aplicaciones desarrolladas para diferentes idiomas y regiones.

La codificación de caracteres recomendada para los sistemas modernos es UTF-8, ya que permite representar correctamente caracteres, símbolos y alfabetos utilizados en distintos idiomas, favoreciendo la interoperabilidad y la internacionalización de las aplicaciones.

Por su parte, el cotejamiento define las reglas que PostgreSQL emplea para comparar y ordenar cadenas de texto. Su configuración influye directamente en el comportamiento de las búsquedas, los filtros y los resultados de las consultas que involucran información textual. Las características de estos parámetros se describen a continuación:

- **Codificación de caracteres (character encoding).** Define la forma en que los caracteres se almacenan y representan dentro de la base de datos. La codificación UTF-8 es la opción recomendada, ya que admite caracteres de múltiples idiomas y garantiza la correcta representación de símbolos especiales.
- **Cotejamiento (collation).** Establece las reglas utilizadas para ordenar y comparar cadenas de texto. Determina, por ejemplo, si las comparaciones

distinguen entre letras mayúsculas y minúsculas y cómo se procesan caracteres propios de determinados idiomas, como las vocales con tilde o la letra ñ en español.

La configuración adecuada de estos parámetros evita inconsistencias en las consultas, garantiza resultados de ordenamiento acordes con el idioma de la aplicación y contribuye al desarrollo de soluciones informáticas preparadas para operar en entornos internacionales.

4.3. Restricciones de integridad y relaciones físicas

Una base de datos relacional no constituye únicamente un repositorio de información, sino un sistema capaz de garantizar la calidad y consistencia de los datos almacenados. Para lograrlo, PostgreSQL incorpora restricciones de integridad (`constraints`), las cuales establecen reglas que el motor verifica automáticamente durante las operaciones de inserción y actualización. Estas restricciones impiden el almacenamiento de información incompleta, duplicada o inconsistente, fortaleciendo la confiabilidad de las aplicaciones y preservando la integridad del modelo de datos.

Implementación de restricciones a nivel de tabla y columna (NOT NULL, UNIQUE y CHECK)

PostgreSQL permite incorporar reglas de validación directamente en la definición de las tablas mediante restricciones que controlan el comportamiento de las columnas y garantizan el cumplimiento de las reglas del negocio (Elmasri y Navathe, 2021). Las restricciones más utilizadas se describen a continuación:

- **NOT NULL.** Impide que una columna almacene valores nulos. Obliga a proporcionar un dato válido durante las operaciones de inserción o actualización.
- **UNIQUE.** Garantiza que los valores de una columna, o de una combinación de columnas, sean únicos dentro de la tabla. Se utiliza para evitar la duplicidad de datos, como números de identificación, direcciones de correo electrónico o códigos de productos.
- **CHECK.** Define una condición lógica que cada registro debe cumplir antes de almacenarse. Permite validar reglas del negocio, por ejemplo, verificar que un precio sea mayor que cero o que una edad pertenezca a un rango determinado.

La incorporación de estas restricciones fortalece la integridad de los datos desde el propio motor de bases de datos, reduciendo la posibilidad de almacenar información que incumpla las reglas definidas por la organización.

Materialización de relaciones lógicas mediante claves primarias (PRIMARY KEY) y claves foráneas (FOREIGN KEY)

La capacidad de relacionar información almacenada en diferentes tablas constituye uno de los principios fundamentales del modelo relacional. Este proceso se basa en el concepto de integridad referencial, mediante el cual PostgreSQL garantiza que las relaciones entre los datos permanezcan consistentes durante todo el ciclo de vida de la base de datos. Para establecer estas relaciones se utilizan dos tipos de restricciones fundamentales:

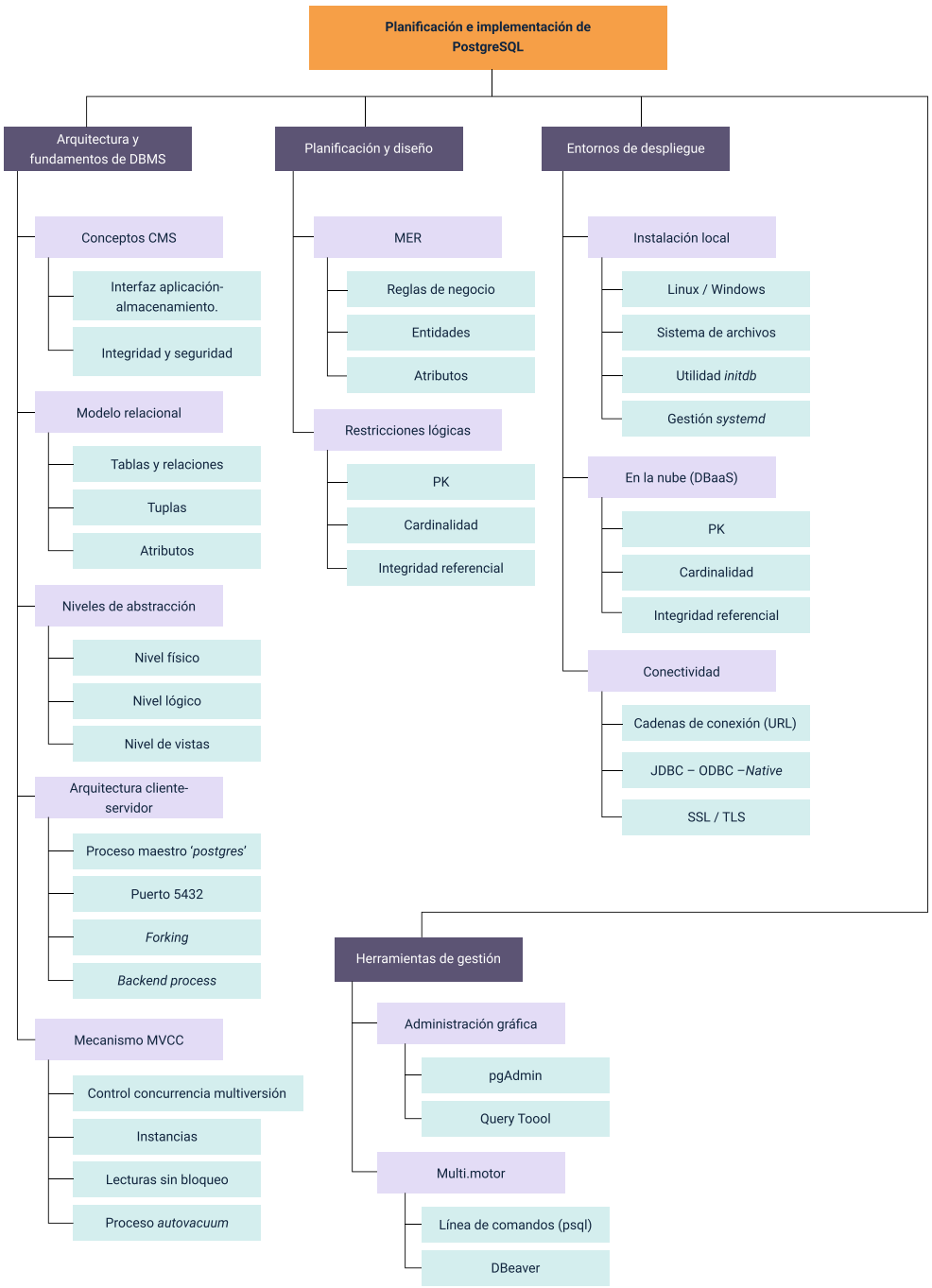
- **PRIMARY KEY.** Identifica de forma única cada registro de una tabla. Esta restricción combina las propiedades NOT NULL y UNIQUE, garantizando que cada fila posea un identificador exclusivo que sirva como referencia para otras tablas.
- **FOREIGN KEY.** Establece una relación entre una tabla y otra mediante una columna cuyos valores deben existir previamente como PRIMARY KEY en la tabla relacionada. Este mecanismo impide registrar información asociada a entidades inexistentes y preserva la integridad referencial del sistema.

Además de validar la existencia de los registros relacionados, las claves foráneas permiten definir acciones automáticas que mantienen la consistencia de la información cuando se modifican o eliminan registros de la tabla principal. Entre las opciones más utilizadas se encuentran ON DELETE CASCADE y ON UPDATE CASCADE, las cuales propagan automáticamente los cambios hacia las tablas relacionadas cuando así lo requiere el diseño de la base de datos.

La implementación adecuada de restricciones de integridad y relaciones físicas garantiza que la información almacenada conserve su consistencia, reduzca la posibilidad de errores y represente fielmente las reglas del negocio definidas durante el diseño conceptual.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo.



Glosario

API: interfaz de Programación de Aplicaciones (Application Programming Interface). Conjunto de reglas, protocolos y herramientas que permite a diferentes aplicaciones o componentes de software comunicarse e intercambiar datos de forma estandarizada entre sí.

Arquitectura cliente-servidor: modelo de diseño de software en el que las tareas se reparten entre los proveedores de recursos o servicios (servidores) y los demandantes (clientes). En PostgreSQL, el cliente envía solicitudes mediante scripts y el servidor las procesa de manera centralizada.

Backend: capa lógica y de acceso a datos de una aplicación de software que opera del lado del servidor. Se encarga de procesar las reglas de negocio, validar la seguridad y realizar las conexiones operativas directas hacia el motor de base de datos.

DBaaS: base de datos como Servicio (Database as a Service). Modelo de computación en la nube donde el proveedor aloja, automatiza, respalda y gestiona la infraestructura física del motor de bases de datos, permitiendo al desarrollador consumirla de forma remota.

Frontend: capa visual, de presentación e interacción de una solución informática con la que el usuario final interactúa de manera directa (por ejemplo, formularios web o aplicaciones móviles que capturan las solicitudes del negocio).

GUI: interfaz Gráfica de Usuario (Graphical User Interface). Entorno visual interactivo compuesto por botones, menús y ventanas que facilita la administración y el

desarrollo de software. Un ejemplo estándar para la gestión de datos es la herramienta web pgAdmin.

Instrucciones SQL: conjunto de comandos estructurados bajo el Lenguaje de Consulta Estructurado (Structured Query Language) utilizados para interactuar con la base de datos. Se dividen en categorías como DDL (Definición de Estructuras) y DML (Manipulación de Datos).

NoSQL: paradigma de almacenamiento de datos no relacional diseñado para gestionar grandes volúmenes de información no estructurada o semiestructurada (como documentos JSON). PostgreSQL integra estas capacidades mediante el uso del tipo de dato nativo JSONB.

ORDBMS: sistema de Gestión de Bases de Datos Relacionales Orientado a Objetos (Object-Relational Database Management System). Es un motor relacional que extiende sus capacidades tradicionales permitiendo definir tipos de datos complejos, herencia de tablas y funciones personalizadas directamente en el servidor.

Proceso en segundo plano: tarea o hilo de ejecución del sistema operativo que se ejecuta de forma silenciosa sin necesidad de interactuar directamente con una interfaz de usuario. En PostgreSQL, procesos como autovacuum operan bajo este esquema para optimizar el servidor.

Sistema operativo: software base de la infraestructura tecnológica que administra los recursos de hardware y proporciona servicios a las aplicaciones. Distribuciones como Linux (Ubuntu, Rocky Linux) son los entornos predilectos para alojar servidores de producción de PostgreSQL.

Tupla: término técnico matemático derivado del modelo relacional original que representa una fila, registro o instancia única de datos almacenada dentro de una estructura bidimensional (tabla).

Referencias bibliográficas

Elmasri, R., & Navathe, S. B. (2021). Fundamentals of database systems. Pearson. https://www.pearson.com/en-us/subject-catalog/p/fundamentals-of-database-systems/P200000003546/9780137502523

Neon. (2026). Neon documentation: Separating storage and compute. https://neon.com/docs/postgres/overview

PostgreSQL Global Development Group. (2026). PostgreSQL 16 documentation: Chapter 18. Server setup and operation. https://www.postgresql.org/docs/16/runtime.html

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2002). Fundamentos de bases de datos. McGraw-Hill. https://biblioteca.univalle.edu.ni/files/original/01aebde3cc06dce33f2538aa2724eb2541cb9473.pdf

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional 06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Jaime Hernán Tejada Llano	Experto temático	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Paola Alexandra Moya	Evaluadora instruccional	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Carlos Julian Ramirez Benitez	Diseñador de contenidos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Robinson Javier Ordoñez Barreiro	Desarrollador full stack	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Alejandro Delgado Acosta	Intérprete lenguaje de señas	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Cristhian Giovanni Gordillo Segura	Intérprete Lenguaje de señas	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Juan Pablo Rojas Polania	Animador y productor multimedia	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Carlos Eduardo Garavito Parada	Animador y productor multimedia	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Maria Carolina Tamayo Lopez	Locución	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
German Acosta Ramos	Locución	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Aixa Natalia Sendoya Fernández	Validador de recursos educativos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila

Nombre	Cargo	Centro de Formación y Regional
Ricardo Oliveros Zambrano	Validador de recursos educativos digitales	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Anyerson Wilfredo Pizo Ossa	Evaluador para contenidos inclusivos y accesibles	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila
Daniel Ricardo Mutis Gómez	Evaluador para contenidos inclusivos y accesibles	Centro Agroempresarial y Desarrollo Pecuario - Regional Huila